

AADG classes: suffix tries, trees and arrays

Norbert Dojer

8.10.2024

1 Suffix tries.

Inspect the code for suffix tries.

1. Consider the problem of searching patterns with a special symbol `#` that can match any character in the text (we assume that `#` does not occur in the text). Implement a method `has_pattern(p)` of the class `SuffixTrie` that checks whether the pattern `p` with a single occurrence of `#` matches a text represented by the suffix trie.
2. Design an algorithm counting the occurrences of a pattern in a suffix trie. What is its complexity? What is the complexity of the analogous algorithm for a suffix tree?

2 Suffix arrays.

Inspect the code for suffix arrays (you can restrict your attention to function `binarySearchSA`).

1. Implement function `search_SA_range(t, sa, p)` that, given text `t`, its suffix array `sa` and a string `p`, computes the ends of the `sa` range containing the offsets of all `t`-suffixes that have a prefix `p`.
2. Assume the above values are already calculated for string `p`. How can they be used to:
 - count the number of occurrences of `p` in `t`?
 - find an occurrence of string `p+'a'` in `t`?